



同濟大學
TONGJI UNIVERSITY

基于 HBase 的二手车交易数据分析实验报告

大数据存储技术小作业

课程名称：_____大数据存储技术_____

学生姓名：_____

学 号：_____

学院专业：_____数据科学与大数据技术_____

指导教师：_____喻剑_____

2026 年 6 月

基于 HBase 的二手车交易数据分析实验报告

1 实验目的与任务要求

本实验围绕二手车交易数据构建 HBase 存储与查询流程。根据作业要求，实验需要完成以下内容：

1. 设计并创建 HBase 表 `car_sales`，包含 `info` 与 `sale` 两个列族。
2. 设计 RowKey，说明其对按城市查询和销售日期范围查询的支持情况。
3. 将 `car_sales_data.csv` 转换为 HBase 可导入的数据格式，并完成普通 `put` 与 `BulkLoad` 两种导入方式。
4. 使用 HBase Filter 完成三类组合查询：北京地区价格 20 万以上、比亚迪品牌且状态为 `pending`、车主姓名以“张”开头。
5. 对 C1001 车辆完成价格多版本写入与历史版本查询。
6. 使用 `checkAndPut` 或等价原子条件写入机制，实现只有 `pending` 状态才能更新为 `sold`。
7. 假设数据扩展到 100 万条，分析 RowKey、Region、写热点和配置参数优化方案。

2 实验环境配置与启动

2.1 Hadoop 与 HBase 运行环境

实验在 AutoDL 云容器中完成。基础环境包括 OpenJDK 8、Hadoop 3.3.6、HBase 2.5.11-hadoop3 以及 Python 3。环境变量配置如下：

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export HBASE_HOME=/opt/hbase
export PATH=$JAVA_HOME/bin:$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$HBASE_HOME/bin:$PATH
```

Hadoop 采用单机伪分布式方式运行，HDFS NameNode、DataNode、Secondary-NameNode 以及 YARN ResourceManager、NodeManager 均已启动。进程检查结果如图 1 所示。

```
root@autodl-container-2254468db4-168e19bc:/opt/hadoop/etc/hadoop# jps
5185 Jps
4012 DataNode
4638 ResourceManager
3887 NameNode
4255 SecondaryNameNode
4783 NodeManager
```

图 1: HDFS 与 YARN 进程启动结果

HBase 配置为分布式模式，数据目录指向 HDFS:

```
hbase.rootdir = hdfs://localhost:9000/hbase
hbase.cluster.distributed = true
hbase.zookeeper.property.dataDir = /root/hbase_zookeeper
```

启动 HBase 后，jps 中出现 HQuorumPeer、HMaster 和 HRegionServer，说明 HBase 基础服务正常运行。

```
root@autodl-container-2254468db4-168e19bc:/opt/hbase/conf# jps
7095 Jps
6072 HQuorumPeer
6280 HMaster
6616 HRegionServer
4012 DataNode
4638 ResourceManager
3887 NameNode
4255 SecondaryNameNode
4783 NodeManager
```

图 2: HBase Master、RegionServer 与 ZooKeeper 进程

进入 HBase Shell 后执行 **status**，结果显示 1 个 active master、1 个 server、0 个 dead server，说明 HBase 可正常对外提供服务。

```
root@autodl-container-2254468db4-168e19bc:/opt/hbase/conf# hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hbase/lib/client-facing-thirdparty/log4j-slf4j-impl-2.17.2.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.5.11-hadoop3, r505b485e462c9cbd318116155b3e37204469085a, Mon Feb 17 17:03:36 PST 2025
Took 0.0012 seconds
hbase:001:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
Took 0.6090 seconds
```

图 3: HBase Shell 状态检查

2.2 Thrift 与 happybase 连接

Python 客户端使用 happybase 连接 HBase，需要启动 HBase Thrift 服务。实验中执行：

```
hbase-daemon.sh start thrift
netstat -tunlp | grep 9090
```

9090 端口监听结果如图 4 所示，说明 Thrift 服务启动成功。

```
root@autodl-container-2254468db4-168e19bc:/opt/hbase/conf# netstat -tunlp | grep 9090
tcp        0      0 0.0.0.0:9090          0.0.0.0:*            LISTEN     7723/java
```

图 4: HBase Thrift 监听 9090 端口

随后再次检查进程，ThriftServer 已出现在 jps 输出中。

```
root@autodl-container-2254468db4-168e19bc:/opt/hbase/conf# jps
8353 Jps
6072 HQuorumPeer
6280 HMaster
6616 HRegionServer
7723 ThriftServer
4012 DataNode
4638 ResourceManager
3887 NameNode
4255 SecondaryNameNode
4783 NodeManager
```

图 5: 包含 ThriftServer 的 HBase 进程列表

happybase 连接测试代码如下：

```
import happybase

conn = happybase.Connection("localhost", 9090, timeout=5000)
print("HBase 连接成功")
print(" 当前表: ", conn.tables())
conn.close()
```

输出显示 HBase 连接成功，说明 Python 程序可以通过 Thrift 访问 HBase。

```
root@autodl-container-2254468db4-168e19bc:/opt/hbase/conf# python - <<'PY'
import happybase

conn = happybase.Connection("localhost", 9090, timeout=5000)

print("HBase 连接成功")
print("当前表: ", conn.tables())

conn.close()
PY
HBase 连接成功
当前表: []
```

图 6: happybase 连接 HBase 测试

3 数据集与表结构设计

3.1 原始数据

原始文件为 car_sales_data.csv，共 1001 行，其中 1 行表头、1000 行二手车交易记录。字段包括：

字段	含义
car_id	车辆编号
city	销售城市
brand	品牌
price	成交价格，单位按数据集记为万元
sale_date	销售日期
owner_name	车主姓名
status	交易状态，包含 pending 与 sold

CSV 前几行如下：

```
car_id,city,brand,price,sale_date,owner_name,status
C1001, 南京, 奥迪,66.9,2024/3/13, 孙颖,sold
C1002, 上海, 奥迪,40.4,2024/4/11, 沈伟,sold
C1003, 重庆, 本田,27.4,2024/5/20, 杨强,pending
C1004, 广州, 奔驰,66.8,2024/12/8, 沈洋,sold
```

3.2 列族设计

作业要求表名为 car_sales，并包含两个列族。实验中设计如下：

列族	列	说明
info	car_id	车辆编号
info	brand	品牌
info	owner_name	车主姓名
info	status	交易状态
sale	city	销售城市
sale	price	成交价格，写入为固定宽度字符串
sale	sale_date	销售日期，格式为 yyyyMMdd

建表时将两个列族的版本数均设置为 3，以满足后续 C1001 价格历史版本查询：

```
create 'car_sales',
  {NAME => 'info', VERSIONS => 3},
  {NAME => 'sale', VERSIONS => 3}

describe 'car_sales'
```

建表与表结构检查结果如图 7 所示。

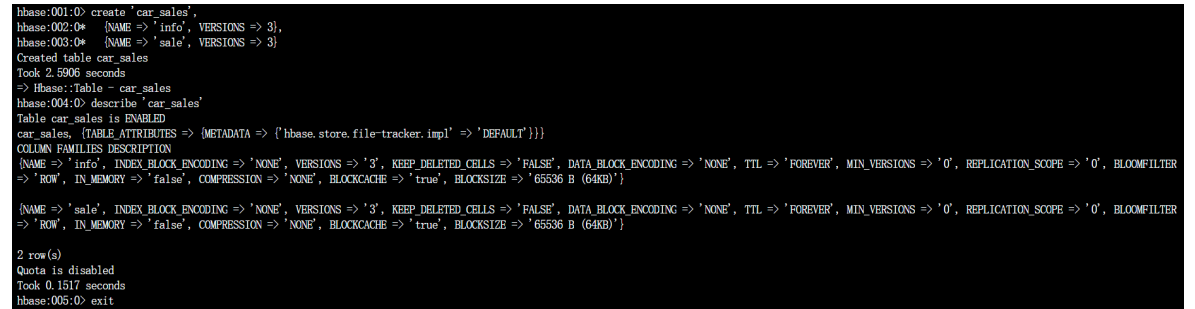


图 7: 创建 car_sales 表并查看列族版本配置

3.3 RowKey 设计

本实验主表采用的 RowKey 为：

```
city#yyyyMMdd#car_id
```

例如：

```
南京 #20240313#C1001
上海 #20240211#C1052
```

该 RowKey 优先服务按城市查询，并兼顾同一城市内的日期范围查询。具体优点如下：

1. 城市放在第一段，同一城市的数据在 HBase 字典序中连续存储，适合用 Prefix-Filter 或 STARTROW/STOPROW 查询某城市数据。
2. 日期放在第二段，在同一城市内部按销售日期排序，适合查询某城市的日期范围，例如 北京 #20240101 到 北京 #20240401。
3. car_id 放在第三段，保证同一城市、同一日期下车辆记录仍能唯一定位。

该设计的不足是全局日期范围查询不够直接。因为 sale_date 不是 RowKey 的第一段，如果要查询所有城市某个日期范围，需要分别扫描多个城市前缀，或者借助按日期组织的二级索引表优化。本文在性能分析部分给出 car_sales_by_date 的 RowKey 设计，用于补足全局日期范围查询能力。

4 数据预处理与普通 put 导入

4.1 CSV 转 TSV

BulkLoad 使用 ImportTsv 生成 HFile，因此先将 CSV 转为 TSV，并把 RowKey 放在第一列。转换脚本为 csv_to_tsv.py，核心逻辑如下：

```
def format_date(date_str):
    for fmt in ("%Y/%m/%d", "%Y-%m-%d"):
        try:
            return datetime.strptime(date_str.strip(), fmt).strftime("%Y%m%d")
        except ValueError:
            pass

def format_price(price):
    return f"{float(price):010.2f}"

def make_rowkey(row):
    return f'{row["city"].strip()}#{format_date(row["sale_date"])}#{row["car_id"].strip()}'
```

价格被格式化为固定宽度字符串，例如 0000020.00，这样 HBase Shell 中可以使用二进制字符串比较完成价格过滤。转换后的 TSV 前几行如图 8 所示。

```
root@autodl-container-2254468db4-168e19bc:~/car_sales_homework# head -5 car_sales_data.tsv
南京#20240313#C1001    C1001    奥迪    孙颖    sold    南京    0000066.90    20240313
上海#20240411#C1002    C1002    奥迪    沈伟    sold    上海    0000040.40    20240411
重庆#20240520#C1003    C1003    本田    杨强    pending    重庆    0000027.40    20240520
广州#20241208#C1004    C1004    奔驰    沈洋    sold    广州    0000066.80    20241208
西安#20240305#C1005    C1005    特斯拉    郭敏    sold    西安    0000037.40    20240305
```

图 8: 转换后的 TSV 文件预览，第一列为 RowKey

4.2 普通 put 导入

普通 put 使用 `normal_put.py` 完成。脚本通过 `happybase` 连接 HBase，将 CSV 中每一行转为一个 HBase row，并写入两个列族：

```
with table.batch(batch_size=1000) as batch:
    for row in reader:
        rowkey = make_rowkey(row)
        batch.put(rowkey, {
            b"info:car_id": row["car_id"].strip().encode("utf-8"),
            b"info:brand": row["brand"].strip().encode("utf-8"),
            b"info:owner_name": row["owner_name"].strip().encode("utf-8"),
            b"info:status": row["status"].strip().encode("utf-8"),
            b"sale:city": row["city"].strip().encode("utf-8"),
            b"sale:price": format_price(row["price"]).encode("utf-8"),
            b"sale:sale_date": format_date(row["sale_date"]).encode("utf-8"),
        })
```

执行命令为：

```
python normal_put.py
```

实验结果显示普通 put 插入 1000 条记录，耗时 0.5141 秒。

```
root@autodl-container-2254468db4-168e19bc:~/car_sales_homework# python normal_put.py
普通 put 插入记录数: 1000
普通 put 插入耗时: 0.5141 秒
```

图 9: 普通 put 导入 1000 条记录的耗时

导入后使用 HBase Shell 执行：

```
scan 'car_sales', {LIMIT => 5}
```

可以看到 5 条记录及其列族、列和值。HBase Shell 对中文按 UTF-8 字节进行转义显示，例如上海显示为 `\xE4\xB8...`，这不影响实际存储。

```
hbase:'001:0' scan 'car_sales', {LIMIT => 5}
ROW
\xE4\xB8\xA8\xE6\xB5\xB7#20240126#C1156
column=info:brand, timestamp=2026-06-05T01:55:37.254, value=\xE5\xA5\x94\x99\xA9\xB0
column=info:car_id, timestamp=2026-06-05T01:55:37.254, value=C1156
column=info:owner_name, timestamp=2026-06-05T01:55:37.254, value=\xE9\x83\x91\xE9\xB9\xBF
column=info:status, timestamp=2026-06-05T01:55:37.254, value=sold
column=sale:city, timestamp=2026-06-05T01:55:37.254, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T01:55:37.254, value=00000094.50
column=sale:sale_date, timestamp=2026-06-05T01:55:37.254, value=20240126
column=info:brand, timestamp=2026-06-05T01:55:37.176, value=\xE5\xA4\xA7\xE4\xBC\x97
column=info:car_id, timestamp=2026-06-05T01:55:37.176, value=C1052
column=info:owner_name, timestamp=2026-06-05T01:55:37.176, value=\xE5\xBC\xA0\xE9\xA2\x96
column=info:status, timestamp=2026-06-05T01:55:37.176, value=pending
column=sale:city, timestamp=2026-06-05T01:55:37.176, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T01:55:37.176, value=00000021.10
column=sale:sale_date, timestamp=2026-06-05T01:55:37.176, value=20240211
column=info:brand, timestamp=2026-06-05T01:55:37.349, value=\xE5\xA4\xA7\xE4\xBC\x97
column=info:car_id, timestamp=2026-06-05T01:55:37.349, value=C1666
column=info:owner_name, timestamp=2026-06-05T01:55:37.349, value=\xE6\x9E\x97\xE6\xB4\xB8
column=info:status, timestamp=2026-06-05T01:55:37.349, value=sold
column=sale:city, timestamp=2026-06-05T01:55:37.349, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T01:55:37.349, value=00000021.90
column=sale:sale_date, timestamp=2026-06-05T01:55:37.349, value=20240219
column=info:brand, timestamp=2026-06-05T01:55:37.349, value=\xE7\x89\xB9\xE6\x96\xAF\xE6\xB8\x99
column=info:car_id, timestamp=2026-06-05T01:55:37.349, value=C1626
column=info:owner_name, timestamp=2026-06-05T01:55:37.349, value=\xE5\x90\xB4\xE9\xA2\x96
column=info:status, timestamp=2026-06-05T01:55:37.349, value=pending
column=sale:city, timestamp=2026-06-05T01:55:37.349, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T01:55:37.349, value=00000037.60
column=sale:sale_date, timestamp=2026-06-05T01:55:37.349, value=20240222
column=info:brand, timestamp=2026-06-05T01:55:37.349, value=\xE8\x94\x9A\xE6\x9D\xA5
column=info:car_id, timestamp=2026-06-05T01:55:37.349, value=C1650
column=info:owner_name, timestamp=2026-06-05T01:55:37.349, value=\xE9\x83\xAD\xE6\x95\xBF
column=info:status, timestamp=2026-06-05T01:55:37.349, value=pending
column=sale:city, timestamp=2026-06-05T01:55:37.349, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T01:55:37.349, value=00000044.90
column=sale:sale_date, timestamp=2026-06-05T01:55:37.349, value=20240303
5 row(s)
Took 0.4838 seconds.
```

图 10: 普通 put 后 scan 前 5 条记录

5 BulkLoad 批量导入

5.1 生成 HFile

为避免普通 put 数据影响 BulkLoad 验证，BulkLoad 前先重建表：

```
disable 'car_sales'
drop 'car_sales'

create 'car_sales',
  {NAME => 'info', VERSIONS => 3},
  {NAME => 'sale', VERSIONS => 3}
```

然后上传 TSV 到 HDFS:

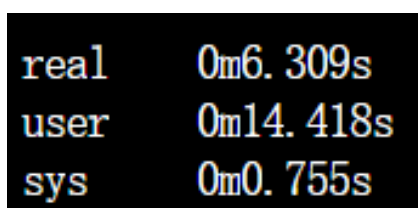
```
hdfs dfs -rm -r -f /user/root/car_sales_input
hdfs dfs -mkdir -p /user/root/car_sales_input
hdfs dfs -put -f car_sales_data.tsv /user/root/car_sales_input/
```

使用 ImportTsv 以 local 模式生成 HFile:

```
COLS="HBASE_ROW_KEY,info:car_id,info:brand"
COLS="${COLS},info:owner_name,info:status,sale:city"
COLS="${COLS},sale:price,sale:sale_date"

time hbase org.apache.hadoop.hbase.mapreduce.ImportTsv \
  -Dmapreduce.framework.name=local \
  -Dimporttsv.separator='${t}' \
  -Dimporttsv.columns="$COLS" \
  -Dimporttsv.bulk.output=/tmp/car_sales_hfiles \
  car_sales /user/root/car_sales_input/car_sales_data.tsv
```

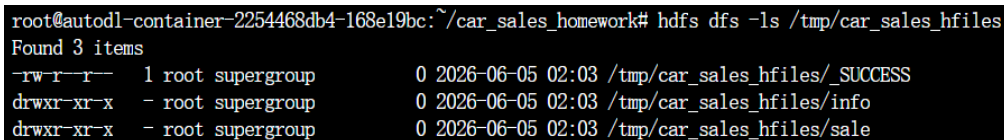
生成 HFile 阶段耗时约 6.309 秒。



```
real    0m6.309s
user    0m14.418s
sys     0m0.755s
```

图 11: ImportTsv 生成 HFile 的耗时

查看 HDFS 输出目录, 结果包含 _SUCCESS、info 与 sale, 说明两个列族的 HFile 已生成。



```
root@autodl-container-2254468db4-168e19bc:~/car_sales_homework# hdfs dfs -ls /tmp/car_sales_hfiles
Found 3 items
-rw-r--r-- 1 root supergroup      0 2026-06-05 02:03 /tmp/car_sales_hfiles/_SUCCESS
drwxr-xr-x - root supergroup      0 2026-06-05 02:03 /tmp/car_sales_hfiles/info
drwxr-xr-x - root supergroup      0 2026-06-05 02:03 /tmp/car_sales_hfiles/sale
```

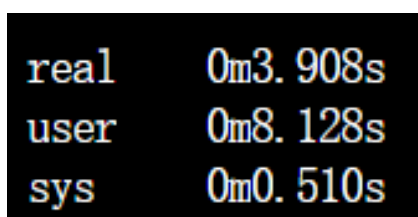
图 12: BulkLoad 输出目录中的 HFile 结构

5.2 执行 completebulkload

执行 BulkLoad:

```
time hbase completebulkload /tmp/car_sales_hfiles car_sales
```

加载阶段耗时约 3.908 秒。



```
real    0m3.908s
user    0m8.128s
sys     0m0.510s
```

图 13: completebulkload 加载 HFile 的耗时

完成后执行：

```
count 'car_sales'
scan 'car_sales', {LIMIT => 5}
```

结果显示 1000 row(s)，并能扫描到表中前 5 条记录，说明 BulkLoad 导入成功。

```
hbase:001:0> count 'car_sales'
Current count: 1000, row: \xE9\x9D\x92\xE5\xB2\x9B#20241230#C1474
1000 row(s)
Took 0.7784 seconds
=> 1000
hbase:002:0> scan 'car_sales', {LIMIT => 5}
ROW
COLUMN+CELL
column=info:brand, timestamp=2026-06-05T02:03:54.362, value=\xE5\xA5\x94\xE9\xA9\xB0
column=info:car_id, timestamp=2026-06-05T02:03:54.362, value=C1156
column=info:owner_name, timestamp=2026-06-05T02:03:54.362, value=\xE9\x83\x91\xE9\xB9\x8F
column=info:status, timestamp=2026-06-05T02:03:54.362, value=sold
column=sale:city, timestamp=2026-06-05T02:03:54.362, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T02:03:54.362, value=0000084.50
column=sale:sale_date, timestamp=2026-06-05T02:03:54.362, value=20240126
column=info:brand, timestamp=2026-06-05T02:03:54.362, value=\xE5\xA4\xA7\xE4\xBC\x97
column=info:car_id, timestamp=2026-06-05T02:03:54.362, value=C1052
column=info:owner_name, timestamp=2026-06-05T02:03:54.362, value=\xE5\xBC\xA0\xE9\xA2\x96
column=info:status, timestamp=2026-06-05T02:03:54.362, value=pending
column=sale:city, timestamp=2026-06-05T02:03:54.362, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T02:03:54.362, value=0000021.10
column=sale:sale_date, timestamp=2026-06-05T02:03:54.362, value=20240211
column=info:brand, timestamp=2026-06-05T02:03:54.362, value=\xE5\xA4\xA7\xE4\xBC\x97
column=info:car_id, timestamp=2026-06-05T02:03:54.362, value=C1666
column=info:owner_name, timestamp=2026-06-05T02:03:54.362, value=\xE6\x9E\x97\xE6\xB4\x8B
column=info:status, timestamp=2026-06-05T02:03:54.362, value=sold
column=sale:city, timestamp=2026-06-05T02:03:54.362, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T02:03:54.362, value=0000021.90
column=sale:sale_date, timestamp=2026-06-05T02:03:54.362, value=20240219
column=info:brand, timestamp=2026-06-05T02:03:54.362, value=\xE7\x89\xB9\x96\xAF\xE6\x8B\x89
column=info:car_id, timestamp=2026-06-05T02:03:54.362, value=C1626
column=info:owner_name, timestamp=2026-06-05T02:03:54.362, value=\xE5\x90\xB4\xE9\xA2\x96
column=info:status, timestamp=2026-06-05T02:03:54.362, value=pending
column=sale:city, timestamp=2026-06-05T02:03:54.362, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T02:03:54.362, value=0000037.60
column=sale:sale_date, timestamp=2026-06-05T02:03:54.362, value=20240222
column=info:brand, timestamp=2026-06-05T02:03:54.362, value=\xE8\x94\x9A\xE6\x9D\xA5
column=info:car_id, timestamp=2026-06-05T02:03:54.362, value=C1650
column=info:owner_name, timestamp=2026-06-05T02:03:54.362, value=\xE9\x83\xAD\xE6\x95\x8F
column=info:status, timestamp=2026-06-05T02:03:54.362, value=pending
column=sale:city, timestamp=2026-06-05T02:03:54.362, value=\xE4\xB8\xA8\xE6\xB5\xB7
column=sale:price, timestamp=2026-06-05T02:03:54.362, value=0000044.90
column=sale:sale_date, timestamp=2026-06-05T02:03:54.362, value=20240303
5 row(s)
Took 0.0475 seconds
hbase:003:0> exit
```

图 14: BulkLoad 后 count 与 scan 验证

5.3 普通 put 与 BulkLoad 对比

方式	数据量	实验耗时	结果
普通 put	1000 条	0.5141 秒	成功写入 1000 条
ImportTsv 生成 HFile	1000 条	6.309 秒	生成 info、sale HFile
completebulkload	1000 条	3.908 秒	加载成功
BulkLoad 命令阶段合计	1000 条	约 10.217 秒	count 验证为 1000

本实验数据规模只有 1000 条，且 ImportTsv 使用 local 模式，MapReduce/工具启动开销占比较大，因此普通 put 的实测耗时更短。BulkLoad 的优势主要体现在大规模离线导入：它先生成 HBase 内部 HFile，再直接加载到 Region 中，减少逐条 RPC、

WAL 和 MemStore 写入压力。若数据规模扩展到百万级或更高，BulkLoad 通常比逐条 put 更稳定、更适合批量装载。

6 HBase Filter 查询

过滤查询使用 HBase Shell 完成，实际输出保存在 filter_log.txt 中。三条查询均限制输出前 10 行，便于在报告中展示结果。

6.1 北京地区价格在 20 万以上

命令如下：

```
scan 'car_sales', {  
  FILTER => "(PrefixFilter('北京#') AND  
    SingleColumnValueFilter('sale','price',>=,'binary:0000020.00'))",  
  LIMIT => 10  
}
```

其中 PrefixFilter('北京#') 利用 RowKey 的城市前缀，只保留北京地区记录；SingleColumnValueFilter 对 sale:price 做比较，筛选价格大于等于 0000020.00 的车辆。由于价格在写入时已经格式化为固定宽度字符串，字符串字典序与数值大小一致。

日志结果显示该查询返回 10 行，耗时 0.6498 秒，前几条包括：

```
北京 #20240103#C1125 price=0000047.20 status=sold  
北京 #20240109#C1775 price=0000027.50 status=sold  
北京 #20240112#C1481 price=0000068.80 status=pending
```

6.2 比亚迪品牌且状态为 pending

命令如下：

```
scan 'car_sales', {  
  FILTER => "(SingleColumnValueFilter('info','brand',=,'binary:比亚迪') AND  
    SingleColumnValueFilter('info','status',=,'binary:pending'))",  
  LIMIT => 10  
}
```

这里两个 SingleColumnValueFilter 通过 AND 组合，要求 info:brand 等于“比亚迪”，且 info:status 等于 pending。日志结果返回 10 行，耗时 0.0525 秒，前几条包括：

```
上海 #20241111#C1601 brand= 比亚迪 status=pending  
南京 #20240724#C1868 brand= 比亚迪 status=pending  
南京 #20240818#C1486 brand= 比亚迪 status=pending
```

6.3 车主姓名以“张”开头

命令如下：

```
scan 'car_sales', {  
  FILTER => "SingleColumnValueFilter('info','owner_name',=,'regexstring:~ 张.*')",  
  LIMIT => 10  
}
```

`regexstring:~ 张.*` 使用正则表达式匹配姓名，`~` 表示字符串开头，因此只返回车主姓名以“张”开头的记录。日志结果返回 10 行，耗时 0.0393 秒，前几条包括：

```
上海 #20240211#C1052  owner_name= 张颖  
上海 #20240305#C1908  owner_name= 张超  
上海 #20240310#C1432  owner_name= 张涛
```

7 版本控制实验

作业要求对 C1001 车辆的价格从 12.5 改为 13.0，再改为 14.0，并查询该车辆的 3 个历史价格版本。由于 RowKey 包含城市和日期，先通过 `find_keys.py` 找到 C1001 的 RowKey：

```
C1001_ROWKEY = 南京 #20240313#C1001
```

随后在 HBase Shell 中执行三次 `put`：

```
put 'car_sales', '南京 #20240313#C1001', 'sale:price', '0000012.50'  
put 'car_sales', '南京 #20240313#C1001', 'sale:price', '0000013.00'  
put 'car_sales', '南京 #20240313#C1001', 'sale:price', '0000014.00'
```

查询最近 3 个版本：

```
get 'car_sales', '南京 #20240313#C1001', {COLUMN => 'sale:price', VERSIONS => 3}
```

输出中保留了同一 Cell 的 3 个版本，按时间戳从新到旧分别是 0000014.00、0000013.00、0000012.50。

```
hbase:001:0> put 'car_sales', '南京#20240313#C1001', 'sale:price', '0000012.50'  
Took 0.5119 seconds  
hbase:002:0> put 'car_sales', '南京#20240313#C1001', 'sale:price', '0000013.00'  
Took 0.0050 seconds  
hbase:003:0> put 'car_sales', '南京#20240313#C1001', 'sale:price', '0000014.00'  
Took 0.0051 seconds  
hbase:004:0> get 'car_sales', '南京#20240313#C1001', {COLUMN => 'sale:price', VERSIONS => 3}  
COLUMN                                CELL  
sale:price                             timestamp=2026-06-05T02:13:23.356, value=0000014.00  
sale:price                             timestamp=2026-06-05T02:13:17.519, value=0000013.00  
sale:price                             timestamp=2026-06-05T02:13:05.174, value=0000012.50  
1 row(s)  
Took 0.0476 seconds
```

图 15: C1001 车辆 sale:price 的 3 个历史版本

该实验说明 HBase 的数据模型以 RowKey、列族、列限定符和时间戳共同定位 Cell。只要列族 VERSIONS 设置足够大，对同一个 Cell 多次写入时，HBase 会保留多个历史版本。本实验建表时将 sale 列族的版本数设置为 3，因此可以查询到最近三次价格变化。

8 原子操作 checkAndPut

8.1 实现方式

作业要求使用 checkAndPut 实现“只有当车辆状态为 pending 时，才允许将状态改为 sold”。本次实验环境中的 HBase Shell 不支持 checkandput 命令，happybase 的 Table 对象也没有直接可用的 check_and_put 方法。因此，实验最终使用 HBase Java API 的 CheckAndMutate 实现等价的原子条件写入。

核心 Java 代码位于 CheckAndPutDemo.java:

```
Put put = new Put(Bytes.toBytes(rowkey));
put.addColumn(FAMILY, QUALIFIER, Bytes.toBytes(newStatus));

CheckAndMutate checkAndMutate = CheckAndMutate
    .newBuilder(Bytes.toBytes(rowkey))
    .ifEquals(FAMILY, QUALIFIER, Bytes.toBytes(expectedStatus))
    .build(put);

CheckAndMutateResult result = table.checkAndMutate(checkAndMutate);
return result.isSuccess();
```

编译与运行命令为:

```
javac -encoding UTF-8 -cp "$(hbase classpath)" CheckAndPutDemo.java
java -cp ".:$(hbase classpath)" CheckAndPutDemo
```

8.2 成功与失败场景

实验选取两个 RowKey:

```
PENDING_ROWKEY = 上海 #20240211#C1052
SOLD_ROWKEY    = 上海 #20240126#C1156
```

运行结果如图 16 所示:

```
===== 成功场景 =====  
操作前状态: 上海#20240211#C1052 pending  
checkAndPut 返回: true  
操作后状态: 上海#20240211#C1052 sold  
  
===== 失败场景 =====  
操作前状态: 上海#20240126#C1156 sold  
checkAndPut 返回: false  
操作后状态: 上海#20240126#C1156 sold
```

图 16: CheckAndMutate 原子条件写入成功与失败场景

成功场景中，上海 #20240211#C1052 原状态为 pending，满足条件，因此更新为 sold，返回 true。失败场景中，上海 #20240126#C1156 原状态已经是 sold，不满足期望值 pending，因此不执行更新，返回 false，状态保持 sold。

原子操作在二手车交易场景中非常重要。如果多个客户端同时尝试售出同一辆车，普通 put 可能造成状态覆盖或重复售卖。CheckAndMutate 将“检查当前值”和“执行更新”作为一个原子过程提交到 RegionServer，只有当前状态仍为 pending 时才会写入 sold，从而避免并发条件竞争。

9 小规模性能分析与优化方案

作业要求假设 car_sales 将扩展到 100 万条数据，且出现两个问题：按 sale_date 范围扫描 5000 条记录变慢，以及写入速度不稳定、单个 Region 写入慢。本节从 RowKey、Region 分布、写热点和 HBase 配置角度分析。

9.1 当前 RowKey 对查询的影响

当前 RowKey 为 city#yyyyMMdd#car_id。它对城市查询友好，因为城市是第一段。例如查询北京数据时，只需扫描 北京 # 前缀范围。它对“某城市内的日期范围查询”也较友好，因为日期是第二段，同一城市内的记录按日期连续排序。

但它不适合“全局销售日期范围查询”。如果查询 2024 年 3 月所有城市的销售记录，由于 RowKey 第一段是城市，HBase 无法从一个连续的日期起点扫描到日期终点，而是需要分别扫描北京、上海、南京等多个城市前缀，或者全表扫描后再过滤 sale:sale_date。当数据达到 100 万条时，这会产生较多无效扫描，导致查询变慢。

9.2 写热点与 Region 问题

写热点主要来自 RowKey 前缀集中和 Region 分布不均。当前 RowKey 以城市开头，如果大量数据集中在北京、上海、广州等少数城市，这些城市前缀的数据可能连续落入同一个或少数几个 Region。写入时这些 Region 的 MemStore、WAL 和 flush 压力较高，可能出现单个 Region 写入慢、整体写入速度不稳定的问题。

另外，若数据导入时接近按城市、日期排序写入，则 RowKey 增长具有局部顺序性，短时间内可能集中写入某些 Region。即使最终数据分布较均匀，导入阶段仍可能出现临时热点。

9.3 优化方案

9.3.1 保留主表并增加日期索引表

主表继续使用 `city#yyyyMMdd#car_id`，服务城市查询和城市内日期范围查询。同时增加日期索引表 `car_sales_by_date`：

```
RowKey = yyyyMMdd#city#car_id
```

该表用于全局销售日期范围查询。查询某一日期范围时，可以直接设置：

```
STARTROW => '20240301'  
STOPROW  => '20240401'
```

这样 HBase 扫描范围与日期范围天然一致，避免跨城市前缀重复扫描。主表与索引表可以在写入时双写，或在离线导入时同时生成两套 HFile。

9.3.2 预分区与 split

为了避免单 Region 承载过多写入，应在建表时预分区，或在热点出现后手动 split。主表可以按城市前缀预分区：

```
create 'car_sales',  
  {NAME => 'info', VERSIONS => 3},  
  {NAME => 'sale', VERSIONS => 3},  
  SPLITS => ['上海#', '北京#', '南京#', '广州#', '成都#', '杭州#', '深圳#', '重庆#',  
            '西安#']
```

如果表已经存在，可对热点 Region 执行：

```
split 'car_sales', '北京 #'  
split 'car_sales', '上海 #'  
split 'car_sales', '广州 #'
```

日期索引表可按月份或季度预分区：

```
create 'car_sales_by_date',
  {NAME => 'info', VERSIONS => 3},
  {NAME => 'sale', VERSIONS => 3},
  SPLITS => ['202403', '202406', '202409', '202412']
```

如果未来写入量继续增长，可以在 RowKey 前增加低位 hash bucket，例如 bucket#city#yyyyMMdd#car_id，将热点城市打散到多个 Region。但这种方式会增加查询复杂度，按城市查询需要扫描多个 bucket，因此更适合写入压力明显高于查询便利性的场景。

9.3.3 HBase 配置参数

可结合机器内存和负载调整以下参数：

参数	优化作用
hbase.hregion.memstore.flush.size	增大单 Region MemStore flush 阈值，减少频繁 flush，提升批量写入稳定性。
hbase.regionserver.global.memstore.size	提高 RegionServer 可用于写缓冲的内存比例，缓解高并发写入时的 flush 压力。
hbase.regionserver.handler.count	增加 RegionServer RPC 处理线程数，提高并发请求处理能力。
hfile.block.cache.size	增加读缓存比例，提高热点查询命中率。
COMPRESSION => 'SNAPPY'	对列族启用 Snappy 压缩，降低磁盘 IO 和网络传输压力。
BloomFilter => 'ROW'	对以 RowKey 定位或前缀查询的场景减少不必要 HFile 读取。

配置调整应结合监控指标进行。若瓶颈是写入 flush 和 compaction，优先关注 MemStore、Region 数量和 HFile 压缩；若瓶颈是范围查询读取，优先优化 RowKey/索引表、BlockCache 和扫描范围。

10 实验结论

本实验完成了基于 HBase 的二手车交易数据建模、导入、查询和优化分析。实验创建了 car_sales 表，设计了 info 与 sale 两个列族，并采用 city#yyyyMMdd#car_id 作为主表 RowKey。该设计能够较好支持按城市查询，以及同一城市内的日期范围查询。

数据导入方面，普通 put 使用 happybase 批量写入 1000 条记录，耗时 0.5141 秒；BulkLoad 通过 CSV 转 TSV、ImportTsv 生成 HFile、completebulkload 加载完成，最终 count 'car_sales' 验证为 1000 行。由于数据量较小且 BulkLoad 使用 local 模式，普通 put 在本实验中更快，但 BulkLoad 在大规模离线导入中更适合。

查询方面，实验使用 HBase Filter 完成了城市加价格、品牌加状态、姓名前缀三类查询。版本控制方面，C1001 车辆的 sale:price 成功保存并查询到 3 个历史版本。原子操作方面，使用 Java API CheckAndMutate 实现了等价 checkAndPut，成功演示了 pending 到 sold 的成功更新，以及 sold 不满足条件时返回 false 的失败场景。

对于百万级扩展，当前 RowKey 在全局日期范围查询上存在不足，并可能因城市前缀集中产生写热点。优化方向包括增加按日期组织的二级索引表、按城市或日期预分区、对热点 Region 执行 split、必要时引入 hash bucket，以及调整 MemStore、BlockCache、RPC handler、压缩等 HBase 参数。

A 截图文件命名说明

原始截图已统一移动到 `figures/` 目录，并按实验含义重命名。正文未使用的 VS Code 插件提示截图也保留在该目录中，便于追溯。

文件名	含义
<code>hadoop_jps_hdfs_yarn_processes.png</code>	HDFS 与 YARN 进程检查
<code>hbase_jps_master_regionserver.png</code>	HBase Master、RegionServer、ZooKeeper 进程检查
<code>hbase_shell_status.png</code>	HBase Shell status 输出
<code>hbase_shell_test_table_crud.png</code>	临时 test 表创建、put、scan、drop 验证
<code>thrift_port_9090_listening.png</code>	Thrift 9090 端口监听
<code>hbase_jps_with_thriftserver.png</code>	含 ThriftServer 的 jps 输出
<code>happybase_connection_test.png</code>	Python happybase 连接测试
<code>hbase_create_car_sales_describe.png</code>	car_sales 建表与 describe 输出
<code>normal_put_1000_records_time.png</code>	普通 put 写入 1000 条记录耗时
<code>normal_put_scan_limit5.png</code>	普通 put 后 scan 前 5 条
<code>tsv_rowkey_preview.png</code>	TSV 转换结果预览
<code>importtsv_hfile_generation_time.png</code>	ImportTsv 生成 HFile 耗时
<code>hfile_output_directories.png</code>	HFile 输出目录结构
<code>completebulkload_time.png</code>	completebulkload 加载耗时
<code>bulkload_count_and_scan.png</code>	BulkLoad 后 count 与 scan 验证
<code>version_control_c1001_price_versions.png</code>	C1001 价格 3 个版本查询
<code>checkandput_success_failure.png</code>	CheckAndMutate 成功与失败场景
<code>vscode_latex_extension_warning.png</code>	VS Code LaTeX 插件提示，非实验结果截图

B 提交文件清单

- `experiment_report.tex`: 本实验报告 LaTeX 源文件。
- `figures/`: 已重命名的截图目录。
- `car_sales_data.csv`: 原始二手车交易数据。
- `car_sales_data.tsv`: BulkLoad 使用的 TSV 文件。
- `normal_put.py`: 普通 put 导入脚本。
- `csv_to_tsv.py`: CSV 转 TSV 脚本。
- `find_keys.py`: 查找 C1001、pending、sold 示例 RowKey 的脚本。

- `CheckAndPutDemo.java`: `CheckAndMutate` 原子条件写入 Java 程序。
- `filter_log.txt`: 三条 Filter 查询的实际 HBase Shell 输出。